

TRABAJO DE FIN DE GRADO · INGENIERÍA DE
TELECOMUNICACIONES

Autenticación Descentralizada basada en Blockchain para el Control de Acceso a Datos

Sustitución de los servidores de identidad centrales (RADIUS / AD) por un sistema de Identidad Autosoberana (SSI) sobre blockchain, con fuente de verdad descentralizada de credenciales y permisos.

Autor: [Nombre del alumno]

Tutor: [Nombre del tutor]

Fecha: Curso 2026-2027 · **Plataforma de desarrollo:** Kali Linux 2026.3

IMPLEMENTACIÓN MVP · RED LOCAL · COSTE CERO

Índice

1. Resumen ejecutivo	3
2. Motivación y problemática	4
3. Objetivos del TFG	5
4. Estado del arte: Identidad Autosoberana	6
5. Decisiones de diseño (DEC-01 ... DEC-09)	7
6. Arquitectura del sistema	8
6.1 Capas lógicas	8
6.2 Flujo de autenticación y autorización	8
7. Módulo 1 — Entorno y blockchain local	9
8. Módulo 2 — Contratos de identidad, revocación y permisos	10
8.1 IdentityRegistry	10
8.2 RevocationRegistry	10
8.3 AccessPolicyRegistry (RBAC)	10
8.4 Story del "momento mágico"	10
9. Módulo 3 — Proxy Validador (diseño)	12
10. Módulo 4 — PostgreSQL y PEP (diseño)	13
11. Módulo 5 — Integración y pruebas (diseño)	14
12. Seguridad en el diseño	15
13. Métricas y evaluación	16
14. Registro de dificultades (errores reales)	17
15. Guía de reproducción	18
16. Trabajo futuro	19
17. Referencias del repositorio	20

Metodología: desarrollo incremental por módulos con ramas Git, criterios de aceptación verificables por módulo y documentación de cada error y su causa raíz (apartado 14), siguiendo la estrategia "primero el plan, luego módulo a módulo".

1. Resumen ejecutivo

Este proyecto implementa un **sistema de autenticación y autorización descentralizado** cuya fuente de verdad para identidades, credenciales y permisos reside en una **blockchain Ethereum local**, en lugar de un servidor central de identidad (RADIUS, Active Directory o la propia tabla `users / roles` de una base de datos). El objetivo práctico es **gobernar el acceso a una base de datos** — quién puede leer, escribir o borrar cada tabla — eliminando el punto único de fallo del modelo centralizado y blindando la autenticación frente a ataques de denegación de servicio (DoS).

El sistema se construye como un **MVP demostrable** sobre una red local de coste cero (sin gas, sin dependencia de internet): usuario → **Proxy Validador** → base de datos. El Proxy es la única puerta: verifica la identidad del usuario (firma digital sobre la petición), comprueba sus permisos contra la blockchain y solo entonces ejecuta la consulta con un usuario de base de datos restringido.

0 €

coste (red local, sin gas)

100 %

cobertura de tests de contratos

3

contratos sólidos compilados

16

tests verdes

Estado actual: Módulos 1 y 2 implementados y verificados (contratos y RBAC completos, cobertura 100 %). Módulos 3–5 definidos con diseño cerrado (secciones 9–11).

2. Motivación y problemática

2.1 El punto único de fallo de la identidad centralizada

Los modelos clásicos de control de acceso — RADIUS en redes cableadas/IP, Active Directory en entornos corporativos, o la tabla de usuarios del propio SGBD — concentran toda la decisión de acceso en **un único servidor**. Esta concentración tiene tres consecuencias estructurales:

- **Disponibilidad:** la caída del servidor de identidad bloquea *toda* la autenticación (la red se queda sin accesos legítimos).
- **Resiliencia ante DoS:** un ataque de denegación de servicio dirigido al servidor de identidad paraliza el sistema completo; el atacante solo necesita robar un punto.
- **Confianza y privacidad:** el servidor central conoce todas las credenciales; un robo de esa base equivale al robo de todas las identidades.

2.2 Por qué blockchain aporta una alternativa

Una blockchain **permissionada o local** aporta tres propiedades a la decisión de acceso:

- **Estado distribuido:** la misma verdad sobre identidades y permisos se replica en varios nodos; no existe un único servidor que derribar.
- **Inmutabilidad y auditoría:** cada concesión, revocación o rotación de clave queda registrada en un ledger que no se puede alterar retroactivamente.
- **Soberanía del usuario:** el portador de la identidad firma sus propias peticiones con su clave privada; el sistema nunca almacena claves de usuario.

Delimitación: el MVP usa una **red local de un nodo** (Hardhat) para demostrar el mecanismo; la mejora a varios nodos (p. ej. Besu IBFT) está prevista en la hoja de ruta (sección 16), sin cambios en los contratos.

3. Objetivos del TFG

Objetivos general y específicos

ID	Objetivo	Cómo se demuestra
OG	Probar que la blockchain puede sustituir al servidor central de identidad para controlar el acceso a datos, sin punto único de fallo.	Arquitectura operativa usuario → proxy → BD gobernada por contratos.
OE1	Definir un modelo de identidad autosoberana (DID + claves de wallet).	DIDs verificables firmando peticiones con claves privadas; rotación de clave autogestionada.
OE2	Implementar RBAC granular en cadena (rol → tabla → operación).	Contrato <code>AccessPolicyRegistry</code> con <code>can(did, tabla, op)</code> .
OE3	Resolver la revocación de accesos en caliente.	Revocación reversible de rol y revocación permanente de identidad, con eventos.
OE4	Diseñar el Proxy que traduzca identidad → SQL sin exponer credenciales de BD.	Arquitectura sección 6 y diseño Módulo 3/4.
OE5	Medir y comparar el coste de autenticar por blockchain vs. forma tradicional.	Plan de métricas, sección 13.

4. Estado del arte: Identidad Autosoberana (SSI)

El modelo **SSI (Self-Sovereign Identity)**, estandarizado en el W3C, invierte el control de la identidad: el sujeto es el *holder* de sus credenciales y las presenta bajo demanda, sin que un servidor central las custodie. Se apoya en tres piezas:

Trinidad técnica W3C aplicada al proyecto

Pieza	Rol en el papel	Implementación en el MVP
DID (Decentralized Identifier)	Identificador único, permanente y verificable del sujeto.	<code>did:web:tfg:demo:doctorA</code> ; en el registro se almacena <code>keccak256(DID)</code> . (DEC-08)
VC (Verifiable Credential)	Atributo firmado por un emisor (RRHH, IT).	En el MVP el "rol" se modela como una política de organización firmada por el admin (<code>assignRole</code>). Extensión a VC completas prevista (sección 16).
VP (Verifiable Presentation)	Prueba criptográfica de que se posee el atributo.	Firma EIP-191 sobre la petición (payload + nonce + timestamp) con la clave de la wallet.

4.1 Elección de blockchain

Comparativa de alternativas (resumen de DEC-02)

Criterio	Redes públicas (ETH/ Polygon)	Hyperledger Fabric	Local Hardhat / Besu (elegido)
Gas / coste	€ por transacción	Sin moneda (MSP)	Cero real
Dependencia de internet	Sí (RPCs públicos)	No (despliegue privado)	No
Latencia de cumplimiento	12 s (L1) / 2 s (L2)	1–2 s (Raft)	~ms (automining)
Permisado / corporativo	Público	Excelente	Según topología (Besu IBFT)
Ecosistema de desarrollo	Extenso (Solidity)	Go/Java chains	Solidity + Hardhat 3 + ethers

Justificación central para la defensa: el MVP prioriza **cero coste**, **cero dependencia externa** y el **ecosistema Solidity** (mayor reutilización profesional). La red local es una decisión deliberada, no una limitación.

5. Decisiones de diseño (DEC-01 ... DEC-09)

Registro de decisiones (documento vivo)

ID	Decisión	Justificación
DEC-01	Modelo SSI (DID + VC + VP).	El usuario es holder; el sistema no custodia secretos.
DEC-02	Blockchain local Ethereum (Hardhat); no mainnet/testnet.	Gas = 0, sin RPC externo, sin SPOF. Sustituible por Besu (mismo Solidity).
DEC-03	Contratos en Solidity 0.8.34; Proxy en Python/FastAPI.	Estándar de facto + agilidad en verificación criptográfica.
DEC-04	Hardhat 3 (Ignition, profile build).	Despliegues declarativos con estado; TS6 de serie.
DEC-05	En cadena solo hashes y claves públicas; verificación fuera de cadena.	Coste y latencia mínimos; cadena como ledger auditable.
DEC-06	Caché TTL + JWT efímero + eventos para vencer la latencia.	Responde a "¿por qué no RADIUS a 10 ms?" (ver sección 9).
DEC-07	Documentar fases y errores con causa raíz.	Material real para la memoria y trazabilidad.
DEC-08	DID → dirección Ethereum (wallet).	Verificación <code>ecrecover</code> nativa; usa EIP-191 para rotación.
DEC-09	API de roles <code>assignRole</code> / <code>unassignRole</code> .	Evita colisión de ABI con <code>grantRole</code> de OpenZeppelin.

El detalle completo de cada decisión está en `docs/01-decisiones.md` del repositorio.

6. Arquitectura del sistema

6.1 Capas lógicas

CAPA DE USUARIO

Wallet/DID + clave privada + SDK. Nunca se revela la clave; cada petición viaja firmada: POST /query { did, payload(sql, nonce, ts), firma }

CAPA DE ACCESO (PEP) = PROXY VALIDADOR (FastAPI)

- Verifica firma (EIP-191) y anti-replay (nonce + ventana temporal)
- Consulta on-chain (solo en miss de caché): registrado?, revocado?, can(did, tabla, op)?
- Emite sesión JWT efímera (60–120 s) → latencia ~ms en consultas sucesivas
- Escucha eventos de revocación → invalida caché y sesiones (< 2 s)
- Traduce rol → privilegios SQL y ejecuta con usuario de BD restringido

CAPA DE CONSENSO / BLOCKCHAIN (fuente de verdad)

IdentityRegistry · RevocationRegistry · AccessPolicyRegistry
(registro, rotación, revocación, RBAC granular, eventos de auditoría)

CAPA DE DATOS

PostgreSQL 16 (Docker): escucha solo 127.0.0.1; el único usuario con credenciales es el del Proxy (grants mínimos)

6.2 Flujo de autenticación y autorización

```

P1 Usuario: payload = { sql, nonce, ts }; firma = Sign(keccak(payload)) [wallet]
P2 Proxy:   valida ts (±30 s) y nonce (no repetido) [sin gas]
P3 Proxy:   owner de la identidad (caché → on-chain si miss) [view]
P4 Proxy:   ¿isRevoked? ¿can(did, tabla, operación)? [view]
P5 Proxy:   OK → JWT de sesión (rol, did, exp) y ejecución en BD con el
            usuario restringido del rol correspondiente
P6 Proxy:   devuelve filas/error
P7 Async:   suscripción a eventos Revoked / RoleRevokedFor → invalidación
            de caché y de sesiones del DID afectado
  
```

7. Módulo 1 — Entorno y blockchain local

El Módulo 1 establece la base: un **monorepo** con estructura por módulos (`contracts/` `test/` `ignition/` `scripts/` `infra/` `docs/`), el toolchain **Hardhat 3.17** (template `mocha-ethers`) y un primer contrato de humo (`IdentityRegistry` stub) verificado de extremo a extremo contra un nodo local.

Herramientas del entorno (instaladas y verificadas)

Herramienta	Versión	Rol
Node.js / npm	24.19.0 / 11.19.0	Ejecución del toolchain
Hardhat	3.17.0	Framework, red local, tests y cobertura
Hardhat Ignition	3.1.8	Despliegues declarativos con estado
solc	0.8.34	Compilador Solidity (EVM objetivo: Osaka)
ethers.js	6.17	Interacción con contratos
OpenZeppelin	5.6.1	AccessControl, ECDSA, MessageHashUtils

7.1 Verificación del módulo (resultados reales)

```
$ npx hardhat compile      # 3 ficheros Solidity compilados (evm target: osaka)
$ npx hardhat test         # 16 passing

$ npx hardhat node --port 8545
$ npx hardhat ignition deploy ignition/modules/AccessStack.ts --network localhost
  → AccessStackModule#IdentityRegistry      0xe7f1... (nodo local)
  → AccessStackModule#RevocationRegistry    0x9fE4...
  → AccessStackModule#AccessPolicyRegistry  0x5FbD...
```

8. Módulo 2 — Contratos de identidad, revocación y permisos

El Módulo 2 convierte el stub en la **fuentes de verdad** completa. Tres contratos con responsabilidades separadas para responder a tres preguntas distintas: *¿quién eres?*, *¿estás expulsado?* y *¿puedes hacer esto?*.

8.1 IdentityRegistry — identidad y rotación de clave

- **DEC-08:** cada DID (hash) se asocia a una **dirección Ethereum** (clave secp256k1 de la wallet del usuario).
- `registerDID(didHash, owner)`: solo `ADMIN_ROLE`; rechaza duplicados.
- `rotateKey(didHash, newOwner, sig)`: **autogestión** — quien posee la clave ANTIGUA firma (EIP-191/ `eth_sign`) el payload `keccak256(didHash, newOwner)`; el contrato recupera la dirección con `ECDsa.recover`. Sin intervención del admin (soberanía).
- `deactivate(didHash)`: kill-switch administrado e irreversible.
- Vistas sin gas: `isRegistered`, `isDeactivated`, `getOwner`.
- Eventos: `DIDRegistered`, `KeyRotated`, `IdentityDeactivated`.

8.2 RevocationRegistry — revocación irreversible

- `revoke(didHash)`: solo `REVOKER_ROLE` (rol concedible a más operadores por el admin). Emite **Revoked** — el evento que el Proxy escucha para invalidar caché y sesiones.
- `isRevoked(didHash)`: vista sin gas.
- Semántica: **irreversible** (expulsión / clave comprometida). Es la pieza crítica de cualquier sistema descentralizado: aquí la revocación es inmediata, auditable y propagable.

8.3 AccessPolicyRegistry — RBAC descentralizado

- Roles de organización: `ROLE_ADMIN`, `ROLE_WRITER`, `ROLE_VIEWER`, `ROLE_ANALYST`.
- Operaciones como bitmask: `OP_READ=1`, `OP_WRITE=2`, `OP_DELETE=4`.
- `setPolicy(rol, tabla, ops)`: solo admin; define la matriz rol→tabla→operación.
- `assignRole(did, rol)` / `unassignRole(did)`: **reversibles**; emiten `RoleGrantedFor` / `RoleRevokedFor`.
- `can(did, tabla, op)`: **oráculo de autorización**, vista sin gas, usada por el Proxy.
- **DEC-09:** nombres propios para evitar colisión ABI con `AccessControl.grantRole`.

8.4 Story de demostración: el "momento mágico"

Escenario médico sobre el que se apoya la defensa: doctores, analistas y un intruso.

Permisos reales consultados al contrato (salida del seed)

Usuario	Rol	Consulta	Resultado
doctorA	VIEWER	SELECT diagnosticos	PERMITIDO
doctorA	VIEWER	UPDATE diagnosticos	DENEGADO
doctorA	VIEWER	DELETE diagnosticos	DENEGADO
analyst	ANALYST	SELECT ventas	PERMITIDO
analyst	ANALYST	SELECT diagnosticos	DENEGADO
intruso	— (no registrado)	cualquiera	DENEGADO

Momento mágico (en vivo, 2 transacciones):

Doctor A lee diagnosticos	→ true
TX unassignRole(doctorA)	→ (1 transacción)
Doctor A lee diagnosticos AHORA	→ false ← acceso retirado en caliente
TX assignRole(doctorA)	→ (1 transacción)
Doctor A lee diagnosticos DE NUEVO	→ true ← acceso restituido

Verificación del módulo: **16 pruebas Mocha verdes y cobertura 100 % de líneas / statements** en los tres contratos (criterio del plan: > 80 %). Seed idempotente.

9. Módulo 3 — Proxy Validador (diseño cerrado)

Componente central para responder a la pregunta más esperada del tribunal: **"¿una consulta a la blockchain no tarda 2 segundos frente a los 10 ms de RADIUS?"**. La respuesta es arquitectónica: **la blockchain nunca está en el camino crítico**.

Estrategia de latencia del Proxy

Mecanismo	Efecto
Caché TTL (30 s) de estado on-chain	Identidad/rol/revocación verificados una vez, reutilizados N veces (<i>cache hit</i>).
Token de sesión JWT efímero (60–120 s)	Consultas sucesivas del mismo usuario = validación local (~milisegundos).
Vistas sin gas (lecturas)	Los <i>miss</i> de caché no pagan consenso: solo un <code>eth_call</code> .
Listener de eventos <code>Revoked</code>	La revocación se propaga en < 2 s invalidando caché y sesiones.
Fail-closed / fail-open por recurso	Con la cadena caída pero caché caliente, el acceso continúa; decisión política.

9.1 API orientativa

```

POST /auth/verify { did, payload{sql, nonce, ts}, firma } → { veredicto, rol, jws }
POST /query      { jws, sql }                        → filas o error 403/401
POST /admin/revoke { did }                            → invalida caché/sesiones
(admin)
GET /metrics                                           → Prometheus

```

10. Módulo 4 — PostgreSQL en Docker (PEP, diseño)

Elección del tribunal: **PostgreSQL 16 en Docker** como base de datos protegida. La defensa se apoya en **defensa en profundidad**: el Proxy es la puerta lógica y el SGBD la red de seguridad.

Barreras anti-bypass

Barrera	Implementación
Red interna Docker	PostgreSQL escucha solo 127.0.0.1 / red privada; ningún contenedor cliente puede alcanzar el puerto 5432 salvo el Proxy.
Usuario de BD único y restringido	El Proxy usa un usuario con grants mínimos por rol (traducción de la política on-chain); los clientes jamás reciben credenciales de BD.
Validación en el Proxy	Parseo de la query y <i>allowlist</i> de tablas según can(did, tabla, op) .
Consecuencia de error del Proxy	Aunque el traducción fallara, el SGBD rechaza la operación por falta de grant (segunda barrera real).

Cliente(DID+clave) → Proxy(FastAPI, usuario BD restringido) → PostgreSQL 16 :5432 (127.0.0.1)

por firewall/red interna

└─ redes externas: BLOQUEADO

11. Módulo 5 — Integración final y pruebas (diseño)

- **Orquestación:** `docker compose` con nodo blockchain, Proxy, PostgreSQL y cliente de demo; scripts `up.sh`, `demo.sh`, `chaos.sh`.
- **Demostración en directo:** usuario A (lectura) SELECT OK / DELETE fail; usuario B sin rol → todo denegado; **momento mágico:** cambiar permisos en la blockchain y ver el efecto al instante.
- **Pruebas de estrés y destrucción:** flood de `/verify` (locust), medición de CPU/RAM de nodo y Proxy (`docker stats`), y **caída del nodo blockchain** para demostrar que con caché el acceso legítimo continúa y la revocación local se mantiene.
- **Comparativa:** autenticación tradicional de BD vs. blockchain (cálidos/fríos), con gráficas.

12. Seguridad en el diseño

Matriz de amenazas y mitigaciones

Amenaza	Mitigación
Bypass del Proxy (conexión directa a BD)	Red interna Docker + usuario único de BD con grants mínimos (sección 10).
Replay de petición firmada	Nonce (memoria del Proxy) + ventana temporal ± 30 s.
Robo de clave de usuario	Rotación autogestionada (<code>rotateKey</code>) + revocación irreversible de identidad (<code>RevocationRegistry</code>).
Revocación tardía de permisos	JWT corto + invalidación por eventos (< 2 s).
DoS al servidor de identidad	No existe: la verdad está replicada/validable en la capa blockchain; el Proxy es stateless y escalable.
Escritura maliciosa / fuga	Traducción rol $\rightarrow$ grants + validación de query + grants mínimos.
Alteración retroactiva de permisos	Ledger inmutable; cada acción emite evento auditable.

13. Métricas y evaluación

Plan de métricas del Módulo 5

Métrica	Unidad	Objetivo
Latencia de autenticación (frío / caliente / JWT)	ms (p50/p95/p99)	p95 < 150 ms caliente; ~ms con JWT
coste por transacción (gas)	gas / USD	0 en red local (solo escrituras admin)
CPU / RAM del nodo y del Proxy	% / MB	Nodo < 2 vCPU amp; < 2 GB bajo carga
Propagación de la revocación	ms	< 2000
Disponibilidad ante caída de nodo	% / evento	Continuidad con caché caliente
Ratio de acierto de caché	%	> 90

14. Registro de dificultades (errores reales y su resolución)

Cada error fue documentado en el repositorio (docs/errores/) con síntoma, causa raíz y solución. Este registro es material de primera mano para el apartado "Dificultades encontradas" de la memoria, demostrando gestión real del proyecto.

Errores encontrados y resolución

#	Error	Causa raíz	Solución
1	npm no ejecuta <code>postinstall</code> de esbuild	Política <code>allowScripts</code> de npm 11	<code>npm install-scripts approve esbuild</code> (aprobación granular)
2	Test comparaba <code>bytes</code> ethers (hex) con <code>Uint8Array</code>	Los <code>bytes</code> se devuelven como cadena hex	Normalizar con <code>ethers.hexlify()</code> ; convención en todos los tests
3	<code>pkill -f</code> mató el propio shell	El patrón estaba en la línea de comandos del shell	Gestión por PID; verificación de estado real
4	<code>nonce has already been used</code> (ethers)	Automining + caché de nonce del emisor	Nonce explícito antes de cada envío
5	Sigue <code>NONCE_EXPIRED</code> incluso con nonce explícito	<code>getTransactionCount</code> del provider 1 por detrás (caché)	JSON-RPC crudo <code>eth_getTransactionCount(pending)</code>
6	Reincidencia de <code>pkill</code> suicida	Hábito de matar por patrón	Guardar PID al lanzar; <code>kill \$(cat pid)</code>
7	<code>toEthSignedMessageHash</code> inexistente (OZ)	API movida a <code>MessageHashUtils</code> en OZ 5.6	Importar <code>MessageHashUtils</code>
8	ABI ambigua <code>grantRole(bytes32, bytes32)</code>	Colisión con <code>AccessControl</code>	Renombrar a <code>assignRole</code> / <code>unassignRole</code> (DEC-09)
9	<code>signMessage(hex)</code> firmaba el texto, no los bytes	ethers firma <code>string</code> como UTF-8	Firmar <code>ethers.getBytes(...)</code>
10	Matcher <code>.reverted</code> deprecado en HH3	Nueva API de matchers	<code>.revert()</code>

14.1 Lecciones transversales

- **No desactivar las políticas de seguridad** de npm; aprobar el paquete exacto.
- **Conocer los tipos de retorno** de ethers (hex vs. binario) antes de escribir tests.
- **Gestión de procesos por PID**, nunca por patrón que esté en la propia línea de comandos.
- **Preferir JSON-RPC crudo** cuando se necesite un estado exacto y vigente de la red.
- **Releer la documentación de la versión concreta** de cada librería (OZ 5.6, HH3).

15. Guía de reproducción

```
# 1) Dependencias
npm install

# 2) Tests y cobertura
npx hardhat test
npx hardhat test --coverage

# 3) Nodo local persistente
npx hardhat node --port 8545 &

# 4) Despliegue (nodo limpio: borrar ignition/deployments si ya hay)
rm -rf ignition/deployments
npx hardhat ignition deploy ignition/modules/AccessStack.ts --network localhost

# 5) Seed de la demo (idempotente)
npx hardhat run scripts/seed.ts --network localhost
```

Módulos 3–5 (Proxy, PostgreSQL, orquestación) añadirán un `docker compose up` como único comando de la demo final.

16. Trabajo futuro

Líneas de evolución planificadas

Mejora	Beneficio
Varios nodos (Besu IBFT / multi-peer) y consenso real	Elimina también el nodo único como punto de concentración; validación real de tolerancia a fallos.
Credenciales verificables completas (VC firmadas por emisor, ZKPs)	Privacidad selectiva en la presentación de atributos.
Rotación por recuperación (dual-clave, threshold)	Mitiga pérdida de clave única.
Entorno de red virtual (Mininet)	Recupera el caso de uso de red (802.1X) si se desea ampliar el alcance.
Hardware wallet / TPM como custodia de clave	Endurece la soberanía del usuario.

17. Referencias del repositorio

Estructura y documentación

Ruta	Contenido
contracts/	IdentityRegistry.sol , RevocationRegistry.sol , AccessPolicyRegistry.sol
test/	Tests Mocha + Chai (TS) de los tres contratos
ignition/modules/	Módulo de despliegue AccessStack.ts
scripts/	seed.ts (idempotente), lib/deployments.ts
docs/01-decisiones.md	Registro DEC-01...DEC-09
docs/fases/	Memoria de cada módulo (1 y 2 realizados)
docs/errores/	Registro de errores con causa raíz (apartado 14)
AGENTS.md	Contexto de construcción (template Hardhat) para agentes IA

Plataforma: Kali Linux 2026.3 · Node 24 · Hardhat 3.17 · Solidity 0.8.34 · OpenZeppelin 5.6.1 · ethers 6.17 · GitHub Actions: no aplica (sin CI en el repo).